

继承

继承语法

```
#include<iostream>
using namespace std;
//继承的好处：减少重复代码
//语法：class 子类 : 继承方式 父类
//子类也称为派生类
//父类也称为基类

class basepage{
public:
    void header()
    {
        cout<<"登录，注册，首页，公开课"<<endl;
    }

    void footer()
    {
        cout<<"帮助中心，交流合作，站内地图"<<endl;
    }
};

class cpp:public basepage{
public:
    void content()
    {
        cout<<"cpp页面"<<endl;
    }
};

class java:public basepage{
public:
    void content()
    {
        cout<<"java页面"<<endl;
    }
};

class python:public basepage{
public:
    void content()
    {
        cout<<"python页面"<<endl;
    }
};

void test()
{
    cpp cpp;
```

```

    java ja;
    python py;

    cpp.content();
    cpp.header();
    cpp.footer();

    ja.content();
    ja.header();
    ja.footer();

    py.content();
    py.header();
    py.footer();

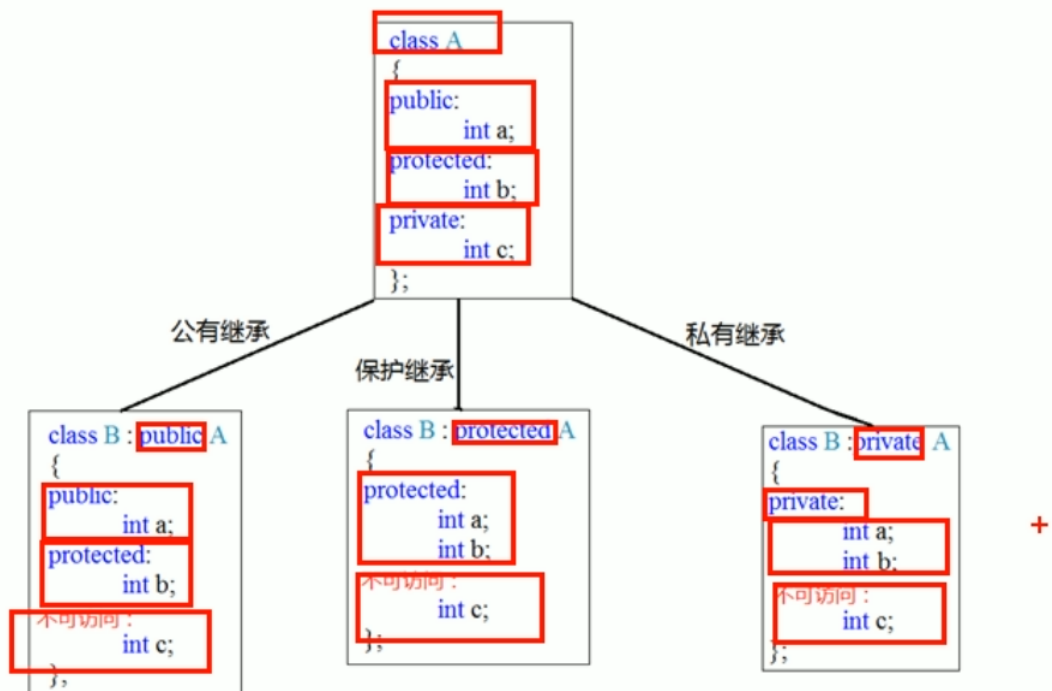
}

int main()
{
    test();

    return 0;
}

```

继承方式



```

#include<iostream>
using namespace std;

```

```

class base1{
public:
    int m_a;

protected:
    int m_b;

private:
    int m_c;
};

//公共继承
class son1:public base1{
public:
    void func()
    {
        m_a=10;//父类中的公共权限成员 到子类中依然是公共权限
        m_b=10;//父类中的保护权限成员 到子类中依然是保护权限
        //m_c=10;//父类中的私有权限成员 子类不能访问
    }
};

//保护继承
class son2:protected base1{
public:
    void func()
    {
        m_a=100;//父类中的公共权限成员 到子类中是保护权限
        m_b=100;//父类中的保护权限成员 到子类中是保护权限
        //m_c=100;//父类中的私有权限成员 子类不能访问
    }
};

//私有继承
class son3:private base1{
public:
    void func()
    {
        m_a=100;//父类中的公共权限成员 到子类中是私有权限
        m_b=100;//父类中的保护权限成员 到子类中是私有权限
        //m_c=100;//父类中的私有权限成员 子类不能访问
    }
};

class grandson:public son3{
public:
    void func()
    {
        //m_a=100;//son3中m_a是私有权限，子类不能访问
        //m_b=100;//son3中m_a是私有权限，子类不能访问
    }
};

```

```

void test1()
{
    son1 s1;
    s1.m_a=100;
    //s1.m_b=100;//到son1中m_b是保护权限，类外访问不到
}

void test2()
{
    son2 s2;
    //s2.m_a=100;//到son2中m_a是保护权限，类外访问不到
    //s2.m_b=100;//到son2中m_b是保护权限，类外访问不到
}

void test3()
{
    son3 s3;
    //s3.m_a=100;//到son3中m_b是私有权限，类外访问不到
    //s3.m_b=100;//到son3中m_b是私有权限，类外访问不到
}

int main()
{
    test1();
    test2();
    test3();

    return 0;
}

```

继承中的对象模型

```

#include<iostream>
using namespace std;

class base{
public:
    int m_a;
    int m_b;
    int m_c;
};

class son:public base{
public:
    int m_d;
};

//父类中所有非静态成员属性都会被子类继承
//父类中私有成员属性也被继承了，但是会被编译器屏蔽，因此子类访问不到父类中的私有成员属性

void test()
{
    cout<<sizeof(son)<<endl;//16
}

```

```
}

int main()
{
    test();

    return 0;
}
```

继承中构造与析构顺序

```
#include<iostream>
using namespace std;

class base{
public:
    base()
    {
        cout<<"base构造函数"<<endl;
    }

    ~base()
    {
        cout<<"base析构函数"<<endl;
    }
};

class son:public base{
public:
    son()
    {
        cout<<"son的构造函数"<<endl;
    }

    ~son()
    {
        cout<<"son的析构函数"<<endl;
    }
};

void test()
{
    son s;
    //构造顺序：先构造父类再构造子类（先有父亲再有儿子）
    //析构顺序：先析构子类再析构父类（白发人送黑发人）
}

int main()
{
    test();

    return 0;
}
```

同名成员处理

```
#include<iostream>
using namespace std;

class base{
public:
    base()
    {
        m_a=100;
    }

    void func()
    {
        cout<<"调用base func()"<<endl;
    }

    void func(int a)
    {
        cout<<"调用base func(int a)"<<endl;
    }

    int m_a;
};

class son:public base{
public:
    son()
    {
        m_a=200;
    }

    void func()
    {
        cout<<"调用son func()"<<endl;
    }

    int m_a;
};

class base1{
public:
    static int m_a;

    static void func()
    {
        cout<<"base func()"<<endl;
    }
};
int base1::m_a=100;//静态成员：类内声明，类外访问

class son1:public base1{
public:
```

```

static int m_a;

static void func()
{
    cout<<"son func()"<<endl;
}
};
int son1::m_a=200;

void test1()
{
    son s;
    cout<<s.m_a<<endl; //访问子类同名成员，直接访问
    cout<<s.base::m_a<<endl; //通过子类对象访问父类中的同名成员，需要加作用域
    //子类对象.父类::父类成员属性
}

void test2()
{
    son s;
    s.func();
    s.base::func();

    //如果子类中出现与父类同名的成员函数，子类的同名函数会隐藏父类中所有同名成员函数
    //如果想访问到父类中被隐藏的同名成员函数，需要加作用域
    //不管咋样访问父类中的同名成员函数一定要加作用域
    //s.func(100);
    s.base::func(100);
}

void test3()
{
    son1 s;
    cout<<s.m_a<<endl;
    cout<<s.base1::m_a<<endl;
    cout<<son1::m_a<<endl; //通过类名访问
    cout<<son1::base1::m_a<<endl; //通过类名访问
}

void test4()
{
    son1 s;
    s.func();
    s.base1::func();
    son1::func(); //通过类名访问
    son1::base1::func(); //通过类名访问
}

int main()
{
    test1();
    test2();
    test3();
    test4();
}

```

```
    return 0;
}
```

多继承语法

```
#include<iostream>
using namespace std;

class base1{
public:
    base1()
    {
        m_a=100;
    }
    int m_a;
};

class base2{
public:
    base2()
    {
        m_a=200;
    }
    int m_a;
};

class son:public base1,public base2{//多继承方式
public:
    son()
    {
        m_c=300;
        m_d=400;
    }
    int m_c;
    int m_d;
};

void test()
{
    cout<<sizeof(son)<<endl;

    son s;
    cout<<s.base1::m_a<<endl;
    cout<<s.base2::m_a<<endl;
}

int main()
{
    test();

    return 0;
}
```


菱形继承

```
#include<iostream>
using namespace std;

class animal{
public:
    int m_age;
};

//利用虚继承解决菱形继承的问题
//class 子类名 : virtual 继承方式 父类名
//animal称为虚基类
class sheep:virtual public animal{};

class camel:virtual public animal{};

class sheepcamel:public sheep,public camel{};

void test()
{
    sheepcamel sp;
    //当菱形继承，两个父类拥有相同数据，需要加以作用域区分
    sp.sheep::m_age=18;
    sp.camel::m_age=28;

    //菱形继承导致数据有两份，资源浪费
    cout<<sp.sheep::m_age<<endl;
    cout<<sp.camel::m_age<<endl;

    //相当于将继承来的两份数据共享了，在内存中只有一个，且以后者为准，所以不需要加作用域访问
    cout<<sp.m_age<<endl;
}

int main()
{
    test();

    return 0;
}
```